# Objects & Property Descriptors

11 min read · Updated 17 Jul 2026 · Topic 12 of 23

Bookmark · Completed

Objects are JavaScript's core data structure, but interviewers rarely stop at "how do I make one." They probe the machinery underneath: every property is really a *descriptor* with hidden attributes (`writable`, `enumerable`, `configurable`), and understanding those flags is what separates candidates who can just use objects from those who understand them.

This lesson covers creation and access, property descriptors, getters/setters, the three levels of immutability (`freeze` / `seal` / `preventExtensions`), enumeration and copying, and the modern `?.` / `??` operators — with the exact traps interviewers use.

## Creating objects

There are four ways to make an object, and interviewers like to hear the trade-offs.

JS Copy

```
const a = { x: 1 };         // object literal – the default, use this
const b = new Object();      // constructor form – verbose, no reason to pr
b.y = 2;
const proto = { greet() { return "hi"; } };
const c = Object.create(proto); // create with an explicit prototype
console.log(a.x, b.y, c.greet(), Object.getPrototypeOf(c) === proto);
// ⇒ 1 2 hi true
```

`Object.create(proto)` is the only one of these that lets you set the prototype directly — that is its whole point. Passing `Object.create(null)` gives you a **prototype-less** object (a clean dictionary with no inherited `toString`, `hasOwnProperty`, etc.), which is handy for maps of untrusted keys.

Constructor functions are the pre-`class` way to stamp out many similar objects; `new` creates a fresh object, binds `this` to it, runs the body, and returns it.

JS Copy

```
function Point(x, y) { this.x = x; this.y = y; }
const p = new Point(3, 4);
console.log(p.x, p.y, p instanceof Point); // ⇒ 3 4 true
```

### Property access: dot vs bracket, and computed keys

Dot notation (`obj.name`) is for fixed, valid-identifier keys. Bracket notation (`obj["name"]`) is required when the key is stored in a variable, is computed at runtime, or isn't a legal identifier (e.g. `"a-b"`, `"data key"`). **Computed keys** `{ [expr]: value }` let you build the key from an expression right in the literal.

JS Copy

```
const key = "dynamic";
const obj = { [key]: 42, "a-b": 1 };
console.log(obj.dynamic, obj["a-b"], obj[key]); // ⇒ 42 1 42
```

All string keys are just strings — `obj[1]` and `obj["1"]` hit the same property, because non-Symbol keys are coerced to strings.

## Property descriptors

Every property is backed by a **descriptor** — a small config object. A *data* property has four attributes:

| Attribute | Meaning |
| --- | --- |
| `value` | the stored value |
| `writable` | can the value be reassigned? |
| `enumerable` | does it show up in `for ... in`, `Object.keys`, `JSON.stringify`, spread? |

| | |
|---|---|
| `configurable` | can the descriptor be changed or the property deleted? |

When you assign a property normally (`obj.x = 1`), all three flags default to `true`. When you create one with `Object.defineProperty`, **every omitted flag defaults to `false`** — a classic gotcha.

JS Copy

```
const o = {};
Object.defineProperty(o, "x", { value: 1 });
console.log(Object.getOwnPropertyDescriptor(o, "x"));
// ⇒ { value: 1, writable: false, enumerable: false, configurable: false
o.y = 2; // plain assignment
console.log(Object.getOwnPropertyDescriptor(o, "y"));
// ⇒ { value: 2, writable: true, enumerable: true, configurable: true }
```

## A read-only, non-enumerable property

`Object.defineProperty` is how you build "system" fields — an id that can't be reassigned and doesn't leak into serialization or iteration.

JS Copy

```
const user = {};
Object.defineProperty(user, "id", {
  value: 101,
  writable: false,
  enumerable: false,
  configurable: false,
});
user.id = 999;                    // silently ignored (non-strict mode)
console.log(user.id);             // ⇒ 101
console.log(Object.keys(user));   // ⇒ []   (non-enumerable, hidden)
console.log(JSON.stringify(user)); // ⇒ {}  (non-enumerable, not serializ
```

The write "failing silently" is mode-dependent: in a non-strict context the assignment is a no-op, but in **strict mode** it throws.

JS Copy

```
"use strict";
const user = {};
Object.defineProperty(user, "id", { value: 101, writable: false });
try {
  user.id = 999;
} catch (e) {
  console.log(e.name + ": " + e.message);
  // ⇒ TypeError: Cannot assign to read only property 'id' of object '#<O
}
```

## Getters & setters

An *accessor* property replaces `value` / `writable` with a `get` and/or `set` function. It looks like a normal property to callers but runs code on read/write — perfect for computed or validated fields.

JS Copy

```
const temp = {
  _c: 0,
  get celsius() { return this._c; },
  set celsius(v) { this._c = v; },
  get fahrenheit() { return this._c * 9 / 5 + 32; },
  set fahrenheit(f) { this._c = (f - 32) * 5 / 9; },
};
temp.celsius = 25;
console.log(temp.fahrenheit); // ⇒ 77
temp.fahrenheit = 212;
console.log(temp.celsius);    // ⇒ 100
```

`fahrenheit` stores nothing — it derives from `_c` on every access. Note the descriptor of an accessor has `get` / `set` instead of `value` / `writable`, so you can't mix the two

## Immutability: freeze vs seal vs preventExtensions

Three built-ins lock objects down to different degrees. The table is a frequent whiteboard question:

| Method | Add props | Delete props | Reassign existing | Reco |
|---|---|---|---|---|
| `Object.preventExtensions` | ✗ | ✓ | ✓ | ✓ |
| `Object.seal` | ✗ | ✗ | ✓ | ✗ |
| `Object.freeze` | ✗ | ✗ | ✗ | ✗ |

`freeze` is the strongest: nothing can be added, removed, reassigned, or reconfigured.

JS Copy

```
const f = Object.freeze({ a: 1 });
f.a = 2; f.b = 3; delete f.a;   // all no-ops (throw in strict mode)
console.log(f, Object.isFrozen(f)); // ⇒ { a: 1 } true
```

`seal` still lets you **change existing values** — it only forbids adding/removing keys and reconfiguring.

JS Copy

```
const s = Object.seal({ a: 1 });
s.a = 2;     // allowed — still writable
s.b = 3;     // blocked — cannot add
delete s.a;  // blocked — cannot remove
console.log(s, Object.isSealed(s)); // ⇒ { a: 2 } true
```

`preventExtensions` is the loosest: only *adding* new properties is blocked; you can still edit, delete, and reconfigure existing ones.

```
const pe = Object.preventExtensions({ a: 1 });
pe.a = 2;   // allowed (still writable)
pe.b = 3;   // blocked (cannot add a new prop)
console.log(pe, Object.isExtensible(pe));       // ⇒ { a: 2 } false
console.log(Object.isSealed(pe), Object.isFrozen(pe)); // ⇒ false false
```

### Freeze is shallow — the #1 trap

`Object.freeze` only locks the object's **own top-level** properties. A nested object it points to is still fully mutable.

JS Copy

```
const cfg = Object.freeze({ nested: { debug: false } });
cfg.nested.debug = true;          // mutates! the inner object was never fr
console.log(cfg.nested.debug);    // ⇒ true
```

For true immutability you recursively freeze:

JS Copy

```
function deepFreeze(obj) {
  for (const key of Object.keys(obj)) {
    const val = obj[key];
    if (val && typeof val === "object") deepFreeze(val);
  }
  return Object.freeze(obj);
}
const cfg = deepFreeze({ nested: { debug: false } });
cfg.nested.debug = true;
console.log(cfg.nested.debug); // ⇒ false  (now genuinely locked)
```

## Enumerating properties

The methods differ on two axes: **own vs inherited**, and whether they respect **enumerable**.

- `Object.getOwnPropertyNames` — own keys including non-enumerable ones.

JS Copy

```
const base = { inherited: 1 };
const obj = Object.create(base);
obj.own = 2;
Object.defineProperty(obj, "hidden", { value: 3, enumerable: false });

const forIn = [];
for (const k in obj) forIn.push(k);
console.log(forIn);                // ⇒ [ 'own', 'inherited' ]  ← inherited
console.log(Object.keys(obj));     // ⇒ [ 'own' ]
console.log(Object.values(obj));  // ⇒ [ 2 ]
console.log(Object.entries(obj));// ⇒ [ [ 'own', 2 ] ]
```

Neither `for ... in` nor `Object.keys` sees `hidden`, because it's non-enumerable. To filter `for ... in` down to own keys, use a `hasOwn` check.

### hasOwnProperty vs Object.hasOwn

`obj.hasOwnProperty(key)` tests own-vs-inherited, but it can break: if the object has its own property literally named `hasOwnProperty`, or was made with `Object.create(null)` (no prototype), the method call throws. `Object.hasOwn(obj, key)` (ES2022) is the safe, modern replacement.

JS Copy

```
const dict = Object.create(null);
dict.hasOwnProperty = "oops";  // shadows the method; dict.hasOwnProperty(
console.log(Object.hasOwn(dict, "hasOwnProperty")); // ⇒ true  (safe)
console.log(Object.prototype.hasOwnProperty.call(dict, "hasOwnProperty"));
```

## Copying objects

Both spread ( `{ ...obj }` ) and `Object.assign(target, ...sources)` do a **shallow** copy: top-level properties are copied, but nested objects are shared by reference.

JS Copy

```
const original = { a: 1, nested: { z: 9 } };
const copy = { ...original };
copy.a = 2;             // independent — top level was copied
copy.nested.z = 100;    // shared! both see it
console.log(original.a, original.nested.z); // ⇒ 1 100

const merged = Object.assign({}, original, { a: 5 });
console.log(merged.a, merged.nested === original.nested); // ⇒ 5 true (sa
```

For a genuine deep copy, use `structuredClone` (built into modern runtimes) — it clones nested structures, `Map` / `Set` / `Date`, and cyclic references. It does **not** copy functions or class prototypes.

JS Copy

```
const deep = structuredClone(original);
deep.nested.z = -1;
console.log(original.nested.z, deep.nested.z); // ⇒ 100 -1  (fully indepe
```

The old `JSON.parse(JSON.stringify(obj))` trick also deep-copies, but silently drops `undefined`, functions, and `Symbol`s, and turns `Date` into a string — prefer `structuredClone`.

## Optional chaining `?.` and nullish coalescing `??`

`?.` short-circuits to `undefined` the moment a link in the chain is `null` or `undefined`, instead of throwing "Cannot read properties of undefined." It works for property access ( `a?.b` ), calls ( `fn?.()` ), and dynamic keys ( `a?.[k]` ).

JS Copy

```
console.log(user.account?.email);  // ⇒ undefined  (no throw)
console.log(user.getName?.());     // ⇒ undefined  (method may not exist)
```

`??` returns the right side only when the left is `null` or `undefined` — unlike `||`, it does **not** treat `0`, `""`, or `false` as "missing." This is the crucial difference interviewers test.

JS Copy

```
const count = 0;
console.log(count || 10); // ⇒ 10  (0 is falsy — wrong for a real "0 item
console.log(count ?? 10); // ⇒ 0   (0 is a real value)

const flag = "";
console.log(flag || "default"); // ⇒ default
console.log(flag ?? "default"); // ⇒ ""  (empty string is a valid value)
```

Together, `user.account?.email ?? "no email"` reads a possibly-missing deep field and supplies a fallback only for a genuine absence.

## Common interview questions

**Q: What's the difference between `Object.freeze`, `Object.seal`, and `Object.preventExtensions`?**
`preventExtensions` blocks only adding new properties. `seal` also blocks deleting and reconfiguring, but existing values stay writable. `freeze` is `seal` plus making all values read-only. All three are shallow.

**Q: What are the four property descriptor attributes, and how do defaults differ between assignment and `defineProperty`?**
`value`, `writable`, `enumerable`, `configurable`. Plain assignment sets the three booleans to `true`; `Object.defineProperty` defaults every omitted flag to `false`.

**Q: How do you make a property that's hidden from `Object.keys` and `JSON.stringify`?**

**Q: `for ... in` vs `Object.keys` — when would they differ?**
`for ... in` includes inherited enumerable keys; `Object.keys` returns only own enumerable keys. They differ whenever the object has an enumerable property on its prototype chain.

**Q: When would you use `??` instead of `||`?**
When `0`, `""`, or `false` are valid values and only `null`/`undefined` should trigger the fallback — e.g. a default for a count, a toggle, or a form field.

**Q: What's the difference between `Object.hasOwn` and `hasOwnProperty`?**
Both test own-vs-inherited, but `hasOwnProperty` breaks on `Object.create(null)` objects or when a property named `hasOwnProperty` shadows the method. `Object.hasOwn(obj, key)` is the robust ES2022 replacement.

## Gotchas / traps

**Freeze is shallow.** `Object.freeze` locks only the top level; nested objects remain mutable. Deep-freeze recursively or use `structuredClone` to snapshot.

`for ... in` **walks the prototype chain.** It enumerates inherited enumerable keys too, so iterating a dictionary can surprise you. Guard with `Object.hasOwn(obj, key)` or use `Object.keys`.

**Spread and `Object.assign` are shallow.** `{ ...obj }` copies top-level values only; nested objects are shared references. Mutating `copy.nested.x` also mutates the original.

`defineProperty` **defaults to `false`.** Omitting `writable`/`enumerable`/`configurable` creates a read-only, hidden, locked property — often not what you intended.

**Writes to read-only / frozen props fail differently by mode.** Silently ignored in sloppy mode, `TypeError` in strict mode (and inside ES modules, which are always strict).

JS Copy

```
const o = Object.freeze({ a: 1 });
try { o.a = 2; } catch (e) { console.log(e.name); } // ⇒ TypeError
```

**Integer-like keys reorder.** Object keys iterate as: integer-like keys in ascending numeric order first, then string keys in insertion order (Symbols last). So insertion order isn't always preserved.

JS Copy

```
const weird = { b: 1, 2: 1, a: 1, 1: 1 };
console.log(Object.keys(weird)); // ⇒ [ '1', '2', 'b', 'a' ]
```

`?.` **only guards** `null` **/** `undefined`**.** It does not catch other errors, and `a?.b.c` still throws if `a.b` exists but `a.b.c`'s access fails — the `?.` only short-circuits at the `?.` link, not the whole chain.

## QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
const cfg = Object.freeze({ mode: "prod", nested: { debug: false } });
cfg.mode = "dev";
cfg.nested.debug = true;
console.log(cfg.mode, cfg.nested.debug);
```

- ◯ dev true
- ◯ dev false
- ◯ prod true
- ◯ prod false

2. What does this log?

JAVASCRIPT Copy

```
const base = { inherited: 1 };
const obj = Object.create(base);
obj.own = 2;
Object.defineProperty(obj, "hidden", { value: 3, enumerable: false });
const keys = [];
for (const k in obj) keys.push(k);
console.log(keys);
```

- ◯ `[ 'own', 'hidden' ]`
- ◯ `[ 'own' ]`
- ◯ `[ 'own', 'inherited', 'hidden' ]`
- ◯ `[ 'own', 'inherited' ]`

Check answer

3. How do the writable/enumerable/configurable flags default when you create a property with plain assignment (obj.x = 1) versus with Object.defineProperty?

- ◯ Both default all three flags to true.
- ◯ Plain assignment defaults all three to true; Object.defineProperty defaults every omitted flag to false.
- ◯ Both default all three flags to false.
- ◯ Plain assignment defaults them to false; Object.defineProperty defaults them to true.

Previous
Higher-Order Functions & Functional Patterns

Next
The Prototype Chain & Inheritance

## LEARN MORE

### ARTICLES & DOCS

- Object.defineProperty (MDN)
  MDN
- Getters (MDN)
  MDN

### VIDEOS




- Getters and setters in JavaScript
  YouTube




- Getters and setters explained
  YouTube




- Object.defineProperty with get/set
  YouTube

Hand-picked references for this lesson.